V1 Traşlama Algoritması — Teknik Rapor

1. Amaç ve Kapsam

V1 algoritması, beyin BT görüntülerinde uzman maskeleme sonrası kalan artık kafatası kemiği (halo) piksellerinin temizlenmesi için geliştirilmiş **kısıtsız ray-marching** tabanlı bir tıraşlama yöntemidir. Algoritma, görüntünün dört kenarından (sol, sağ, üst, alt) içeriye doğru ışın gönderir; arka planı atlar, parlak kafatası piksellerini silerek beyin dokusuna ulaştığında durur.

Pipeline içindeki konumu: pHash + IoU tabanlı duplicate maske eleme adımından **sonra**, GT karşılaştırması ve metrik hesabından **önce**.

---

2. Girdi ve Çıktı

Öğe	Değer
Girdi klasörü	`SiyahVeKaymaTemizBrainMaskUyg/`
Çıktı klasörü	`Traşlama V1/Önce Tekrarsız Sonra Traşlama V1/`
Görüntü formatı	PNG, 8-bit grayscale, 512×512
Çıktı 1	`cleaned_png/` — tıraşlanmış görüntüler
Çıktı 2	`halo_mask_png/` — silinen halo bölge maskeleri
Çıktı 3	`png_slice_statistics.xlsx` — dilim başına istatistik tablosu

Görüntü I/O için `PIL` tercih edilmiştir; OpenCV (`cv2`) Türkçe karakter içeren Windows yollarında okuma/yazma hatası verdiğinden tercih dışı bırakılmıştır.

---

3. Eşik Hesaplama (`auto_thresholds`)

Algoritma her dilim için iki eşik hesaplar: arka plan eşiği `t_bg` ve beyaz (kafatası) eşiği `t_white`.

3.1. Arka Plan Eşiği (`t_bg`)

```python
```

```
border = np.concatenate([
    img_u8[:5, :].ravel(),    # üst 5 satır
    img_u8[-5:, :].ravel(),   # alt 5 satır
    img_u8[:, :5].ravel(),    # sol 5 sütun
    img_u8[:, -5:].ravel()    # sağ 5 sütun
])
t_bg = int(np.percentile(border, 99.9))
```

Görüntünün dört kenarından 5 piksel kalınlığında çerçeve örneklenir. BT görüntülerinde beyin daima merkezde konumlandığı için kenar piksellerinin neredeyse tamamı hava (arka plan) olarak kabul edilir. P99.9 yüzdelik dilimi alınmasının nedeni, kenardaki gürültü piksellerini (birkaç adet düşük seviyeli artefakt) tolere ederken eşiğin gerçek arka plan değerinde kalmasını sağlamaktır. Bu eşik tipik olarak 0–5 aralığında çıkar.

3.2. Beyaz Eşiği (`t_white`)

python

```
mx = int(img_u8.max())
t_white = int(max(200, 0.90 * mx))
```

Görüntü maksimumunun %90'ı alınır, 200 alt sınır olarak garantilenir. Bu eşik ve üzerindeki pikseller "kafatası kemiği" olarak işaretlenir.

Örnek hesaplamalar:

Görüntü maksimumu	0.90 × max	Taban (200)	Sonuç `t_white`
255	229.5	200	229
240	216.0	200	216
220	198.0	200	200
200	180.0	200	200

Bu formülün önemli özelliği **dilim tipinden bağımsız** olmasıdır: ince uç dilimler ile beyin gövdesini içeren orta dilimlere aynı kural uygulanır. Bu sadeleştirme V1'in yapısal kısıtlamalarının temel kaynağıdır (bkz. Bölüm 6).

---

4. Ray-Marching Tıraşlama (`strip_halo`)

4.1. Genel Prensip

Algoritma dört yönden (Sol→Sağ, Sağ→Sol, Yukarı→Aşağı, Aşağı→Yukarı) görüntüyü tarar. Her

tarama iki fazdan oluşur:

1. **Faz 1 — Arka plan atlama:** Piksel değeri `t_bg` ve altındaysa ilerle.
2. **Faz 2 — Kafatası silme:** Piksel değeri `t_white` ve üzerindeyse sil, maskele, ilerle. Bu koşul bozulduğunda dur.

İki faz arasındaki boşluk (yani `t_bg < piksel < t_white` aralığı) algoritma tarafından dokunulmaz.

4.2. Sol → Sağ Tarama Örneği

```python
for y in range(h):
    x = 0
    while x < w and out[y, x] <= t_bg:
        x += 1
    while x < w and out[y, x] >= t_white:
        halo_mask[y, x] = 255
        out[y, x] = 0
        x += 1
```

Diğer üç yön için yalnızca başlangıç koordinatı ve adım yönü değişir; mantık özdeştir.

4.3. Veri Yapıları

```python
out       = img_u8.copy()              # tıraşlanmış görüntü
halo_mask = np.zeros_like(out, dtype=np.uint8)  # silinen piksel maskesi
```

V1 tasarımının kritik özelliği: **dört yön tek bir `out` dizisini paylaşır.** Bir yön bir pikseli sildiğinde (`out[y,x] = 0`), sonraki yönler bu güncellenmiş diziyi okur. Bu paylaşım Bölüm 6.3'te açıklanan etkileşim sorununu doğurur.

---

5. Çıktı İstatistikleri

Her dilim için `process_file` fonksiyonu aşağıdaki alanları içeren bir kayıt üretir; tüm kayıtlar Excel dosyasında toplanır.

Alan	Açıklama
`filename`	Dilim dosya adı
`height`, `width`	Görüntü boyutları (512×512)
`min`, `max`, `mean`, `std`	Temel piksel istatistikleri
`p95`, `p99`	95. ve 99. yüzdelik dilim değerleri
`t_bg`, `t_white`	Hesaplanan eşikler
`halo_pixels_removed`	Silinen toplam piksel sayısı
`error`	Hata mesajı (varsa)

6. V1'in Yapısal Kısıtlamaları

Üç yapısal sorun, sonraki versiyonlarda (V2 ve sonrası) düzeltilmek üzere belgelenmiştir.

6.1. Derinlik Sınırının Bulunmaması

V1'in Faz 2 döngüsü `while ... and out[y,x] >= t_white` koşuluna bağlıdır; ışının ne kadar ilerlediğine dair bir üst sınır yoktur. Eğer bir tarama hattı boyunca koşul süreklilik kazanırsa (örneğin satırın tamamı parlak veya beyin bulunmuyorsa), algoritma 512 pikselin tamamını kat edebilir ve görüntünün karşı kenarına çıkar. Bu durum ince uç dilimlerde gözlemlenmiş ve **10 adet %100-yenme vakasının** birincil sebebi olmuştur.

6.2. Dilim Tipinden Bağımsız Sabit Eşik

`t_white = max(200, 0.90 × max)` formülü her dilim için aynı şekilde uygulanır. İnce uç dilimlerde beyin dokusu az alan kaplar ve görece yüksek yoğunlukta görünür (tipik olarak 230–245 aralığında). Sabit eşik bu doku ile gerçek kafatası kemiği arasında ayrım yapamaz; doku da silinir. Algoritmanın dilim morfolojisine adapte olamaması bu kısıtlamanın özüdür.

6.3. Paylaşımlı `out` Dizisi ve Yön Etkileşimi

Bir yönün sildiği piksel (`out[y,x] = 0`) sonraki yönün okuduğu değeri değiştirir. Somut bir senaryo:

1. Sol→Sağ taraması `(y=100, x=15)` pikselini siler → `out[100,15] = 0`.
2. Sağ→Sol taraması sağdan ilerler ve `(y=100, x=15)` koordinatına ulaşır.
3. `out[100,15] = 0` değeri `t_bg`'den küçük olduğu için Faz 1 koşulu sağlanır → algoritma bu pikseli arka plan zanneder ve atlar.
4. Foreground'un başlangıç noktasını yanlış tespit eder → ya çift sayım ya da kaçırma oluşur.

7. Performans

Parametre	Değer
Paralelleştirme	`multiprocessing.Pool`, `imap_unordered`
Worker sayısı	`cpu_count() - 1` (test makinesinde 19/20)
Chunksize	4
Toplam işlenen dilim	1253

`imap_unordered` tercih edilmiştir çünkü işlem sırası önemsizdir; sonuçlar Excel kaydından önce `sort_values("filename")` ile sıralanır. Bellek kullanımı düşük tutulmuş, her worker yalnızca tek bir görüntüyü aktif olarak tutar.

---

8. Sonuç

V1 algoritması, beyin BT görüntülerinden artık kafatası piksellerini temizlemek için sade ve hızlı bir taban hat (baseline) yöntem sunar. Dört yönlü ray-marching yaklaşımı ve kenar tabanlı otomatik eşik hesaplama mekanizması algoritmik olarak tutarlıdır. Ancak üç yapısal zaaf — derinlik sınırının olmaması, dilim tipinden bağımsız sabit eşik, paylaşımlı çıktı dizisi — ince uç dilimlerde ve karmaşık morfolojilerde başarımı düşürmektedir. Bu zaaflar, sonraki versiyonlarda yapılan düzeltmelerin gerekçesini oluşturur ve V1'in pipeline içindeki konumu **referans baseline** olarak korunur.